

Unit 2. Design & Loops

2.0 Modularization	108
2.1 Debugging.....	130
2.2 Designing Algorithms.....	145
2.3 Loop Syntax	156
2.4 Loop Output	168
2.5 Loop Design.....	181
2.6 Files.....	190
Unit 2 Practice Test	207
Unit 2 Project : Calendar Program.....	209

2.0 Modularization

Sue is frustrated! She is working on a large project with a couple classmates where there must be a thousand lines of code and three dozen functions. Some functions are huge consisting of a hundred lines of code. Some functions are tiny and don't seem to *do* anything. How can she ever make sense of this mess? If only there was a way to map all the functions in a program and judge how large a function should be.

Objectives

By the end of this class, you will be able to:

- Measure the Cohesion level of a function.
- Measure the degree of Coupling between functions.
- Create a map of a program using a Structure Chart.
- Design programs that exhibit high degrees of modularization.

Prerequisites

Before reading this section, please make sure you are able to:

- Create a function in C++ (Chapter 1.4).
- Pass data into a function using both pass-by-value and pass-by-reference (Chapter 1.4).

Overview

Up to this point, our programs have been relatively manageable in size and complexity. In other words, one could conceivably keep the entire design in your head. Most interesting software problems, however, are far too large and far too complex for this. Very soon, this become so difficult that it is impossible for any one person or even group of people to understand everything. What is to be done?

One of the main techniques we have at our disposal to tame these size and complexity challenges is **modularization**. Modularization is a collection of tools metrics, and techniques that together enable us to reduce large problems into smaller ones.

The first tool we have at our disposal is the Structure Chart. This is a graphical representation of the functions in a program, including how they “talk” to each other. You may have noticed an example of a structure chart in the Unit 1 project.

The second modularization tool is a metric by which we measure the “strength” of a function. This will tell us the degree in which a given function is dedicated to a single task. We call this metric Cohesion.

The third and final modularization tool is a metric by which we measure the complexity of the information interchange between two functions. We call this metric Coupling.

These three tools (Structure Chart, Cohesion, and Coupling) together help a programmer to more effectively modularize a program so it is easier to write the code, easier to fix bugs, and easier to understand.

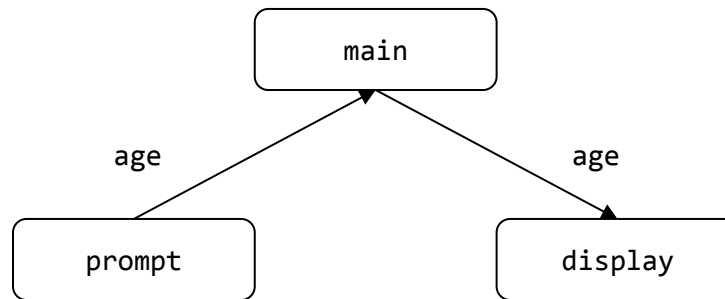
Structure Chart

A Structure Chart is a tool enabling us to design with functions without getting bogged down in the details of what goes on inside the functions. This is basically a graph containing all the functions in the program with lines indicating how the functions get called.

For example, consider a program designed to prompt the user for his age and display a witty message:

```
What is your age: 29
You are 29 again? I was 29 for over a decade!
```

The Structure Chart would be:



There are three components to a Structure Chart: the function, the parameters, and how the functions call each other (program structure).

Functions

Each function in the Structure Chart is represented with a round rectangle. You specify the function by name (remember to camelCase the name as we do with all variable and function names). Since functions are typically verbs, there is typically a verb in the name. In the above example, there are three functions: `main`, `prompt`, and `display`.

Parameters

The second part of a Structure Chart is how information flows between functions. This occurs through parameters as well as through the return mechanism. If a function takes two parameters, then one would expect an arrow to flow into the function with two variables listed. In the above example, the function `prompt` accepts no data from `main` though it sends out a single piece of data: the `age`. Thus the following prototype for `prompt`:

```
int prompt();
```

The next function is `display`. It sends no data back to `main` but accepts a single piece of data, the `age`. Thus one would expect the following prototype for `display`:

```
void display(int age);
```

Program structure

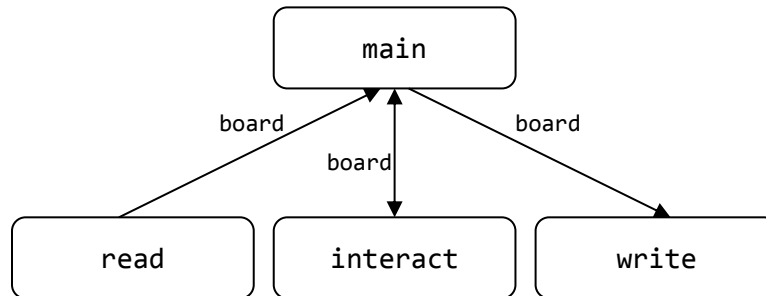
The final part of a Structure Chart is how the functions in a program call each other. Typically, we put `main` on top and, below `main`, all the functions that `main` calls. Note that you can have a single function that is called by more than one function. In this case, arrows will be reaching this function from multiple sources. Please see Project 1 for an example of a Structure Chart.

Designing with Structure Charts

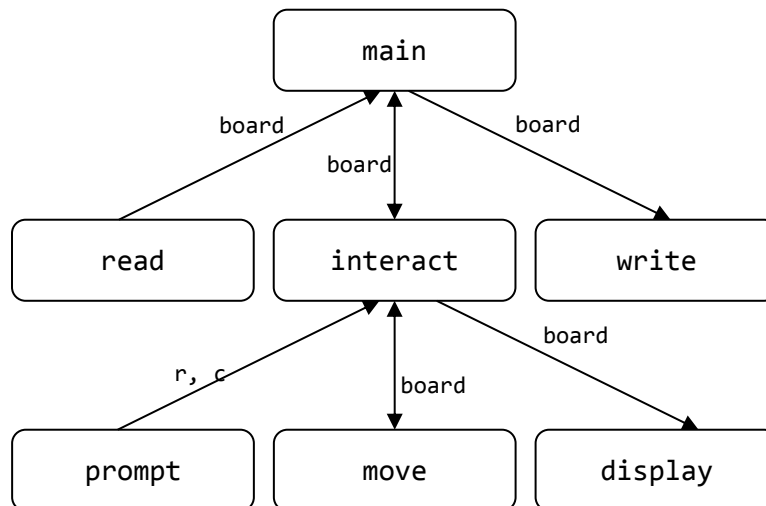
The Structure Chart is often the first step in the design process. Here we answer the big questions of the program:

- What will the program do?
- What are the big parts of the program?
- How will the various parts communicate with each other?

To accomplish this, we start with the top-down approach. We start with very general questions and slowly work to the details. Consider, for example, a program designed to play Tic-Tac-Toe. We would start with a very general design: the program will read a board from a file, allow the user to interact with the board, and then write the board back to the file.

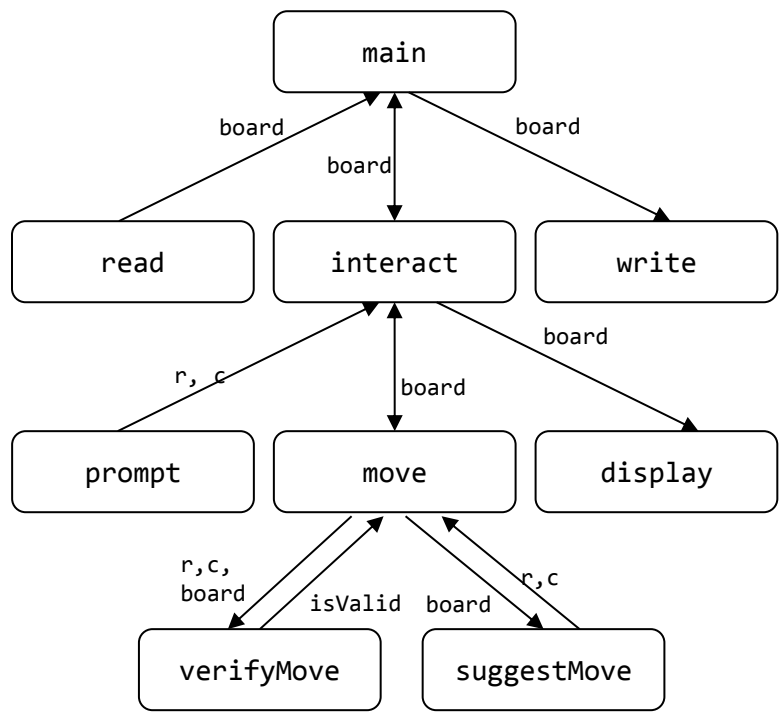


Note how each of the functions is Cohesive (does one thing and one thing only) and has simple Coupling (only one parameter is passed between the functions). That being said, the `interact()` function is probably doing too much work. We will delegate some of that work to three other functions:



Again, each of the functions (`prompt()`, `move()`, and `display()`) are Cohesive and have loose Coupling. However, it appears that the `move()` function is still too complex. While it is still Cohesive, we still might want to delegate some of the work to other functions.

The final Structure Chart for our Tic-Tac-Toe program is the following:

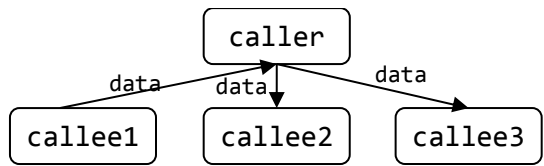


There are a few things to observe about this and all Structure Charts:

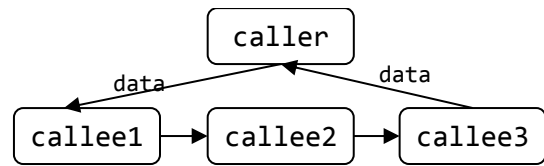
7. We always put `main()` on top. This means that control goes from the top down, rather than following the flow of the arrows. A common mistake new programmers make is to put `main()` in the center of the Structure Chart with arrows extending in all directions. We call this “spider” Structure Charts.
8. There are seldom more than three functions called from a single function. If too many arrows emanate from a given function, then that function is probably doing too much. There are exceptions from this rule-of-thumb of course. One example is when all the child functions do the same type of thing. The Structure Chart from Project 1 is an example. When in doubt, ask yourself if each function is Functionally cohesive.
9. A Structure Chart is a tree; there are no circuits. If a function (`interact()` in the above example) calls another function (`prompt()`), control returns to the caller when the callee is finished.

```

void caller()           // caller will call three functions
{
    int data = callee1(); // first callee1 is called with the return value in data
    callee2(data);        // next data is sent to callee2.
    callee3(data);        // finally data is sent to callee3
}
  
```



Structure Chart of one function calling three in sequence



ERROR: there are no circuits in a Structure Chart! When `callee1()` is finished, data flows back to `caller()`. It cannot flow to `callee2()`

Example 2.0 – Count Factors

Demo

This example will demonstrate how to break a large and complex program into a set of smaller and more manageable functions. When considering how this will be done, the principles of Cohesion and Coupling will be considered. The resulting solution will be presented both as a structure chart and as a set of function prototypes.

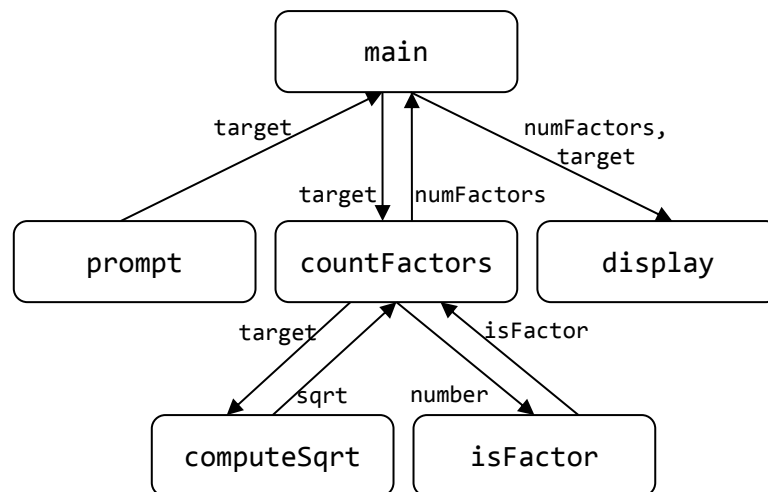
Problem

Write a program to find how many factors a given number has. To accomplish this, it is necessary to enumerate all the values between 1 and the square root of the target number. Each of these values will then need to be checked to see if it evenly divides into the target number.

What number would you like to find the factors for? 16
There are 4 factors in 16.

Solution

The first part of the solution is to identify the functions and how they will call each other. The following structure chart represents one possible solution.



The second part is to convert this map into a set of function prototypes.

```

int main(); // calls prompt, countFactors, & display
int prompt(); // called by main
int countFactors(int target); // called by main, calls computeSqrt, isFactor
void display(int target, int num); // called by main
int computeSqrt(int target); // called by countFactors
bool isFactor(int x, int y); // called by isFactor
  
```

Observe how the structure chart tells you the function names, what parameters go into the functions, and who calls a given function. This allows us to create an outline for the program.

Challenge

As a challenge, try to fill in the functions for this program. The `computeSqrt()` function may look something like this:

```

#include <cmath> // for the SQRT function

int computeSqrt(int target)
{
    return (int)sqrt((double)target);
}
  
```

Cohesion

Recall that Cohesion is a metric by which we measure the “strength” of a function. A more formal definition of Cohesion is:

Cohesion is a measurement of how well a function performs one task.

This definition has two components:

- “a measurement:” Cohesion is a metric, reporting on the quality of one aspect of the design.
- “performs one task:” Cohesion is a property of a single function.

A well-designed function will be completely focused on a single task. There are four different levels of Cohesion (presented from best to worst): Strong, Extraneous, Partial, and Weak.

Strong Cohesion

The strongest and most desirable level of Cohesion is where all the code in a function is directed to one purpose. The formal definition of Strong Cohesion is:

All aspects of a function are directed to perform a single task, and the task is completely represented.

There are two parts to this definition. The first is that the unit of software does nothing extra. Any extra code thrown into a function will forfeit its classification as Strong Cohesion. The second part of the definition is that the task or concept is completely represented. Anything that leaves part of the task undone or relies on the client to complete the work cannot be considered Strong. Of course, every function should strive for Strong cohesion. Observe that it does not matter how simple or complex the task is; it is Strongly Cohesive as long as only that task is being performed.

Consider the following function:

```
/* *****  
 * COMPUTE PAY  
 * Determine an employee's pay based on hourly wage and number of hours worked  
 * ***** */  
float computePay(float hours, float wage)  
{  
    // regular pay  
    if (hours < 40.0)  
        return hours * wage;  
  
    // overtime  
    else  
        return (wage * 40.0) +  
               (wage * 1.5 * (hours - 40.0));  
}
```

Observe how `computePay()` does one thing and one thing only: it computes pay given an employee's hourly wage and number of hours worked. This task is completely accomplished; no other work is needed in order to finish this task.

Extraneous Cohesion

The first weak form of Cohesion is Extraneous. Here exists something unnecessary in the unit of software. A political analogy would be a “rider” on a bill and a sports analogy would be a “bench warmer” on a team. The formal definition of Extraneous Cohesion is:

*At least one part of a function is not directed towards a single task.
However, the principle task is completely represented.*

Streamlined software should have no extraneous functionality. An example of an Extraneous compute-tax function would be one that correctly performs the calculation and then asks the user what to do with the refund. Any time the word “and” is used to completely describe a unit of software, it is probably Extraneous Cohesion.

Consider the following function:

```

/*****
 * COMPUTE PAY
 * Determine an employee's pay based on hourly wage and number of hours worked.
 * This function also displays a warning message if too many hours are worked.
 *****/
float computePay(float hours, float wage)
{
    // display error message if more than the maximum amount of work was done
    if (hours > 60.0)
        cout << "WARNING: Special permission is required to work more than 60 hours.\n";

    // regular pay
    if (hours < 40.0)
        return hours * wage;

    // overtime
    else
        return (wage * 40.0) +
               (wage * 1.5 * (hours - 40.0));
}

```

This function completely accomplishes the task of computing the pay for a worker. Unfortunately, it also does something that is not directly related to the task at hand. In this case, it warns if too much work is reported. As a general rule, a function designed to “compute” should not also “display.”

Any time a function has code that is not directly related to the task at hand, there is a good chance that the function is Extraneous Cohesion (or worse!). Fortunately, the fix is very easy: move the extra code to a more appropriate location.

Partial Cohesion

Another weak form of cohesion is Partial, where a task is left incomplete. In other words, additional data or work needs to be stored or completed elsewhere for the concept or task to be completed. Note that Partial is not below Extraneous in the hierarchy but rather a peer. One does not improve a Partial class by making it Extraneous. Instead, one finished the job it was designed to do. The formal definition of Partial Cohesion is:

All aspects of a function are directed to perform a single task, but the task is not completely represented by the function.

Partial Cohesion is particularly difficult for the client of a system because the onus is on the client to figure out how to finish the job. It is also difficult for the author of the software because, in order to thoroughly test the system, all possible implementations that complete the task needs to be discovered. Any time a description of a system necessitates a detailed description of the context in which it is used, that system is a candidate for Partial cohesion.

Consider the following functions:

```

/*****
 * COMPUTE OVERTIME PAY
 * Determine an employee's pay based on hourly wage and number of hours worked.
 * WARNING: Call computeNormalPay() if hours is less than 40
 *****/
float computeOvertimePay(float hours, float wage)
{
    return (wage * 40.0) +
           (wage * 1.5 * (hours - 40.0));
}

/*****
 * COMPUTE NORMAL PAY
 * Determine an employee's pay based on hourly wage and number of hours worked.
 * WARNING: Call computeOvertimePay() if hours is less than 40
 *****/
float computeNormalPay(float hours, float wage)
{
    return hours * wage;
}

```

Here each of the two functions accomplishes part of the task at hand. Anyone using one of these functions will also have to use the second to get the job done.

There are two ways to fix this problem and make the function(s) Strongly Cohesive: either combine the functionality into a single function or create a wrapper function that calls both of the components:

```

/*****
 * COMPUTE PAY
 * Determine an employee's pay based on hourly wage and number of hours worked
 *****/
float computePay(float hours, float wage)
{
    if (hours < 40.0)
        return computeNormalPay(hours, wage);
    else
        return computeOvertimePay(hours, wage);
}

```

This function is now Strongly Cohesive. It may be desirable to pursue a design like this when there is another part of the program that needs computeNormalPay() or computeOvertimePay() without the other part.

Weak Cohesion

The worst form of cohesion is Weak. One should never design for Weak Cohesion; it is a state that is to be generally avoided. The formal definition of Weak Cohesion is:

*At least one part of a function is not directed towards performing a single task.
Additionally, the task is not completely represented by the function.*

In other words, weak cohesion is a combination of Extraneous and Partial. In theory, one should never come across Weak cohesion. Alas, if only this were true.

Consider the following function:

```
/* *****  
 * COMPUTE PAY  
 * Determine an employee's pay based on hourly wage and number of hours worked.  
 * This function also configures the display for money output.  
 * WARNING: Call computeNormalPay() if hours is less than 40  
 * ***** */  
float computePay(float hours, float wage)  
{  
    // set up the display for money  
    cout.setf(ios::fixed);  
    cout.setf(ios::showpoint);  
    cout.precision(2);  
  
    // regular pay  
    if (hours < 40.0)  
        cout << "ERROR: This only works for hours greater than 40\n";  
  
    // compute overtime pay  
    return (wage * 40.0) +  
           (wage * 1.5 * (hours - 40.0));  
}
```

This function exhibits Extraneous Cohesion. We can tell first because the function comment block contains the word “also.” A subsequent inspection of the code will reveal the code to configure output for money. Since this function does not display anything, the code clearly does not belong here.

This function exhibits Partial Cohesion because it only produces correct output if the employee’s wage is not less than 40 hours. In this case, the program will display an error message on the screen and still produce erroneous output.

Since this function is both Extraneous and Partial, it can be classified as Weakly Cohesive. It appears that the programmer threw code together hoping it would work, rather than properly designed the function. On the surface, it might seem that the best approach from this point is to add the missing functionality and remove the extraneous parts. In practice, a better approach is to start the design process from scratch with Cohesion in mind.

Coupling

Recall that Coupling is a metric of the complexity of the information interchange between two functions. A more formal definition of Coupling is:

Coupling is a measurement of the complexity of the interface between functions.

This definition has two components:

- “a measurement:” Coupling is a metric, reporting on the quality of one aspect of the design.
- “the complexity of the interfaces:” This can be summed up with the question: “how much does the programmer need to know to successfully use and how much does the resulting software need to do?”
- “between:” Coupling fundamentally is a measure of how different parts of a system communicate. It is not a property of an individual function, but rather how it interacts with the rest of the system.

A well-designed interface between functions will be easy to understand and use. There are seven different levels of Coupling (presented from best to worst): Trivial, Encapsulated, Simple, Complex, Document, Interactive, and Superfluous.

Trivial Coupling

Trivial is the weakest or best form of Coupling. Here the client of a unit of software needs to provide no information and receives no information from another unit of software. The formal definition of Trivial Coupling is:

There is no information interchange between functions.

In other words, one unit may instantiate, call, or activate another, but no information is passed. Similarly, no information can be gleaned from the timing of the function call. An example would be a function with no return value and no parameters.

Consider the following function:

```
/* *****  
 * DISPLAY INSTRUCTIONS  
 * Inform the user about the functionality of this program  
 * ***** */  
void displayInstructions()  
{  
    // Inform the user about what this program will do  
    cout << "This program will display the status of your monthly budget.\n"  
        << "The produced report will include both your projected expenditures, "  
        << "as well as what you actually spent last month.\n\n";  
  
    // Inform the user about the type of data that will be requested  
    cout << "To accomplish this, it is necessary to prompt you for several "  
        << "confidential financial details.\n"  
        << "Do not worry, no confidential data will be saved in the process.\n";  
}
```

Note that a function may have no input parameters and have no return type yet still not be Trivially Coupled. Sometimes a hidden global variable is at play, making the function something other than Trivially Coupled.

```
void displayPay()  
{  
    cout << '$' << pay << endl;    // NOT trivially coupled!  
}
```

Encapsulated Coupling

Encapsulated is a very weak form of Coupling defined by all of the parameters being in a trusted and accessible state. In other words, there are logical checks in place to ensure that no invalid data is sent between modules. This level makes no reference to the degree of complexity of the data nor the number of data items passed between modules.

The formal definition of Encapsulated Coupling is:

*All the information exchanged between functions
is in a convenient form and is guaranteed to be in a valid state.*

Note that Encapsulation is an Object-Oriented programming topic and is therefore not a topic for CS 124. That being said, there is a single example of encapsulated data that we have learned about thus far: Boolean data. Since a Boolean variable can have only two states and both are, by definition, valid, it is impossible to pass an invalid Boolean parameter. Enumerations are another validated data-type present in most languages. Here the compiler ensures that the data is always in a valid state. These will be discussed in more detail in CS 165. With modern languages and modern programs, the most common way to achieve the validated status is to use a class whose methods contain checks to guarantee data validity.

Consider the following function:

```

/*****
 * GET IS MALE
 * Prompt the user for his/her gender and return if he/she is male
 *****/
bool getIsMale()
{
    char input;
    cout << "Please select your gender, 'm' for male and 'f' for female: ";
    cin >> input;

    return (input == 'm') || (input == 'M');
}

```

This function takes no input parameters and returns a single Boolean value. The data is in a convenient form and is guaranteed to be valid. Note that Coupling makes no reference to information interchanges between the user and the program; it only concerns itself with information interchange between parts of the program.

Consider this function:

```

/*****
 * DISPLAY CHILD TAX CREDIT STATUS
 * Display to the user the status of their Child Tax Credit
 *****/
void displayChildTaxCreditStatus(bool isEligible)
{
    if (isEligible)
        cout << "You are eligible for the child tax credit.";
    else
        cout << "You cannot claim the child tax credit this year.";
}

```

This is also Encapsulated Coupling because a single Boolean value is passed into the function. From this we can see that though a function may have Trivial input parameters, it is an Encapsulated Coupling if it has a single Encapsulated return value (and vice versa).

Simple Coupling

Another weak form of coupling is Simple, meaning that data can be selected, interpreted, and validated easily. Parameters consisting of simple built-in data-types such as integers or characters are often Simple, assuming that the use of the parameter is easily specified. The formal definition of Simple Coupling is:

All the information exchanged between functions is easy to select, interpret, and validate.

Perhaps this can be best captured with a few questions:

- Can I explain the purpose of this parameter with a few words?
- Is it intuitively obvious what this parameter represents and how it is used?
- Is it easy to validate the parameter with a simple IF statement?

Consider the following function:

```

/*****
 * PROMPT FOR INCOME
 * Prompt the user for his/her monthly/yearly income.
 *****/
double promptForIncome(bool isMonthly)
{
    // different prompt according to the value of isMonthly
    if (isMonthly)
        cout << "Please specify your monthly income: ";
    else
        cout << "Enter your yearly income: ";

    // get the income value
    double income;
    cin >> income;
    return income;
}

```

In this case, the input parameter is a Boolean value. It may seem like this is an example of Encapsulated Coupling. Note, however, that the return value is a `double`. Clearly there are some values which are inappropriate (negative values or numbers which are smaller than a cent), meaning some trivial validation may be required. However, the return value is easy to select, interpret, and validate. This function is thus Simple Coupling.

Consider the following function:

```

/*****
 * ELIGIBLE FOR CHILD TAX CREDIT
 * Return true if eligible for a child tax credit for a given child
 *****/
bool eligibleForChildTaxCredit(double income, int ageChild)
{
    // only eligible if the child is under 17 and income between $2,500 and $200,000
    return (ageChild < 17) && (income >= 2500.00) && (income <= 200000.00);
}

```

Note that the return type makes the function a candidate for Encapsulation Coupling. However, the `income` parameter as well as the `ageChild` parameter meet all the criteria for Simple Coupling. Since the lowest Coupling classification of any parameter (input or output) determines the overall Coupling of the function, this can be classified as Simple Coupling.

Complex Coupling

A tight or bad degree of coupling is Complex. Here the function callers need to know a great deal about the parameters in order to make a connection. The greater knowledge the callers need to have and the more work the callers need to accomplish to make sure the connection is done correctly, the more complex the level of coupling. While it might be worthwhile to enumerate sub-levels of Complex Coupling, it is sufficient to say that all are bad and should be avoided. The formal definition of Complex Coupling is:

At least one piece of information is non-trivial to create, validate, or interpret.

There are many things about a parameter which could cause it to be classified as Complex. A few examples:

- Input that is interpreted as a command, requiring the callee to act in response to the command. Understanding all possible commands is non-trivial.
- Two variables that must be in sync with each other for them to make sense, such as a list of names and a list of addresses. Here the 4th name on the first list corresponds to the 4th address on the second list. Creating and validating these two lists is non-trivial.
- Text that must be in a given format. Passing text consisting of nothing but spaces may constitute an invalid employee last name.

Consider the following function:

```
/******  
 * HANDLE ACTION  
 * Execute one of a collection of actions depending on the "command" parameter  
 *****/  
void handleAction(int command)  
{  
    if (command == 1)  
        displayInstructions();  
    else if (command == 2)  
        openFile();  
    else if (command == 3)  
        playGame();  
    else if (command == 4)  
        exitProgram();  
    else  
        cout << "Invalid or unknown command: " << command << endl;  
}
```

Notice how the parameter is a single integer. This may seem to be a candidate for Simple Coupling. However, the integer is not used in trivial way. Both the caller and the callee share a complex command language where actions are represented with numbers. If the language is expanded or reduced, both the caller and the callee will need to change their code to accommodate the change. This makes the two functions tightly Coupled.

Consider the following function:

```
/******  
 * DISPLAY FULL NAME  
 * Display the user's full name in the format: last name, first name.  
 *****/  
void displayFullName(char nameLast[256], char nameFirst[256])  
{  
    cout << nameLast << ", " << nameFirst;  
}
```

This seemingly simple function is actually incomplete. What if the last name string was empty? What if it contained a newline character or a comma? Dealing with all the formatting needs of a last name (no spaces, only limited punctuation, no newline characters, no numbers) is non-trivial to validate. That makes this function's Coupling level Complex.

Document Coupling

Another tight level of Cohesion is Document. Here data conforming to some language is passed from a producer to a consumer. Note that the producer could be the caller or the callee, depending on the direction of information flow. The important thing to note is that both the producer and the consumer need to fully understand the intricacies of the shared language. The formal definition of Document Coupling is:

At least one piece of information contains a rich language including syntactic and/or semantic rules.

On the surface, this may seem rather difficult to understand and perhaps not an important case worth considering. In practice, however, it is more common than you may think.

In order to demonstrate Document Coupling, it is necessary to use C++ language constructs that will not be introduced for several chapters. Just pay attention to the commands and try to get the jist of the algorithm.

```

/*****
 * EXECUTE ASSEMBLY COMMAND
 * Take action according to an assembly opcode such as "ADD 4"
 *****/
void executeAssemblyCommand(string opcode, int & register)
{
    // get "ADD" from "ADD 4"
    string command = opcode.substr(0, 3);

    // get "4" from "ADD 4"
    string parameter = opcode.substr(4);

    // execute the command.
    if (command == "ADD")
        register += integerFromString(parameter);
    if (command == "SUB")
        register -= integerFromString(parameter);

    ... and so on ...
}

```

The above function needs to understand the complete assembly language and syntax in order to properly handle the input in the opcode parameter. This language certainly has “rich syntatic and sematic rules.”

Another example, this time taken from the Unit 3 project. Consider the game MadLib® where a story has placeholders in which the player of the game will insert his or her own answers to certain questions. The game starts with a raw story similar to the following:

I have a very :adjective pet :animal :.

Here the user will be prompted for an adjective and an animal:

Adjective: silly
Animal: great white shark

The result of this game will be a completed story with the user’s provided prompts:

I have a very silly pet great white shark.

A function interpreting the raw story and generating a completed story will need to exhibit Document Coupling because it will need to understand how to interpret all the prompts.

Interactive Coupling

Interactive Coupling is closely related to Document with an additional component: the information exchange between components is ongoing rather than a single one-time event. Interactive Coupling is usually characterized by a session where conversations begin, the participants react to each other, and conversations are terminated. There are usually syntactic and semantic rules that need to be followed to correctly maintain the conversation. Another way to look at Interactive coupling is a sequence of Document interactions involving maintenance of state by both parties. The formal definition of Interactive Coupling is:

There exists a communication avenue between units of software involving non-trivial dialogs, sessions, or interactions.

It is difficult to show an example of Interactive Coupling using the programming tools presented in CS 124. The main problem is that the two functions exhibiting this interaction need to maintain state. In other words, they both need to keep track of the status of the conversation. To demonstrate this, consider a function called `send()` which sends messages to another function or entity in the system.

```

/*****
 * SEND EMAIL
 * Send an e-mail message using the SMTP protocol
 * Here sendEmail() is demonstrating Simple Coupling whereas send() is Interactive
 *****/
void sendEmail(char data[256])
{
    // initiate the conversation
    char *response = send("HELO");

    // from and to:
    response = send("MAIL FROM:<sender@sourcedomain.com>");
    response = send("RCPT TO:<recipient@destinationdomain.com>");

    // send body of the message
    response = send(data);

    // indicate we are done
    response = send("\n.\n");
    response = send("QUIT");
}

```

The SMTP e-mail protocol involves several many interactions between the client and the server. Notice that all of these interactions occur through the same `send()` function. Here the server responds to messages differently depending on the state of the message. Thus the `send()` function is exhibiting Interactive Coupling.

Superfluous Coupling

The tightest and therefore worst level of coupling is Superfluous. The formal definition of Interactive Coupling is:

At least one piece of data or information is passed between functions unnecessarily.

This is the only level of coupling that makes a reference to the amount of data passed between functions. Data passing between functions is only bad if that data is not necessary. There are two important parts to this definition: what is unnecessary and what is data/information passing. The unnecessary component is completely domain specific. For example, if a function has read/write access to an asset when read-only is required, then the write aspect is unnecessary and the coupling can be classified as superfluous. For example, consider the following function:

```

/*****
 * PROMPT GRADE
 * Prompt the user for his/her GPA
 *****/
float promptGrade(float grade)
{
    cout << "What is your GPA. Please enter a value between 0.0 and 4.0: ";
    cin >> grade;
    return grade;
}

```

On the surface, this appears to be Simple Coupling: the grade parameter needs to be validated before it is used (what if the user selected -1.0 as the GPA?), but that validation can be easily accomplished. However, there is a problem. Data needs to leave this function, but it does not need to enter the function. For some reason, there is an input parameter called grade. This function would be Simple Coupling if grade was a local variable. However, because it is an unnecessary input parameter, this is Superfluous Coupling.

The “passing” component of the definition is a bit more difficult to explain. Consider the range of scope for a language like C++ language (from small to large): block, local variable, one-way parameter (by-value), two-way parameter (by-reference), and global variables. There are legitimate uses for each of these scope levels in many applications. However, if a function utilizes a scope larger than is necessary, then superfluous coupling exists. The degree of superfluosity depends on the number of scope levels beyond that which is necessary that was utilized in a given application. Consider the following function:

```

/*****
 * DISPLAY USER HEIGHT
 * Display the height of the user in convenient units
 *****/
void displayUserHeight(float heightFeet)
{
    // display the height in imperial units
    if (useImperialUnits)
        cout << heightFeet << " feet";

    // display the height in the metric system
    else
        cout << (heightFeet * 0.3048) << " meters";
}

```

This function may appear to be Simple Coupling because there are no output parameters and the single input parameter is easy to verify. However, there is another variable referenced (useImperialUnits) not declared in this function. This function is therefore a global variable, a scope much larger than necessary. The reference to this variable makes the entire function Superfluous Coupling.

Problems

Problem 1

Create a list of the levels of Cohesion from best on the top to worst on the bottom.

- _____
- _____
- _____
- _____

Please see page 117 for a hint.

Problem 2

Classify the form of Cohesion from the following example of code:

```
void displayGPA(float gpa)
{
    // configure the output so the GPA appears correctly
    cout.setf(ios::fixed);
    cout.setf(ios::showpoint);
    cout.precision(1);

    // display the GPA
    cout << gpa;
}
```

Answer:

Please see page 113 for a hint.

Problem 3

Classify the form of Cohesion from the following example of code:

```
float getGPA()
{
    // get the student's GPA
    float gpa;
    cin >> gpa;

    // prompt for the number of credits
    cout << "Please enter the number of credits you are taking this semester: ";
    return gpa;
}
```

Answer:

Please see page 113 for a hint.

Problem 4

Create a list of the levels of Coupling from best on the top to worst on the bottom.

- _____
- _____
- _____
- _____
- _____
- _____
- _____

Please see page 117 for a hint.

Problem 5

Classify the form of Coupling from the following example of code:

```
void displayGreeting()
{
    // prompt for name
    char name[256];
    cout << "What is your name? ";
    cin >> name;

    // display the greeting
    cout << "Hello, " << name << " how are you today?\n";
}
```

Answer:

Please see page 113 for a hint.

Problem 6

Classify the form of Coupling from the following example of code:

```
void setGender()
{
    char input;
    cout << "Are you male? ";
    cin >> input;

    isFemale = (input == 'n') || (input == 'N');
}
```

Answer:

Please see page 113 for a hint.

Problem 7

Match the Cohesion name with the definition:

Strong	At least one part of the function is unnecessary to the main task at hand
Extraneous	Components of the function are irrelevant and parts of the function are missing
Partial	Does one thing completely and one thing only
Weak	The main task of the function is not completely done

Please see page 113 for a hint.

Problem 8

Match the Coupling name with the definition:

Trivial	A dialog exists between functions that involves multiple interactions
Encapsulated	Information is passed between functions that involves a complex language
Simple	At least one parameter is non-trivial to create, validate, or interpret
Complex	All the parameters are in a convenient form and is guaranteed to be in a valid state
Document	At least one parameter is passed between functions unnecessarily
Interactive	There is no information interchange between functions
Superfluous	All the parameters are easy to select, interpret, and validate

Please see page 113 for a hint.

Problem 9

Classify the form of Cohesion from the following example of code:

```
float getGPA()
{
    // set up the display for GPA
    cout.setf(ios::fixed);
    cout.setf(ios::showpoint);
    cout.precision(1);

    // prompt for GPA
    float gpa;
    cout << "Enter your GPA: ";
    cin >> gpa;
    return gpa;
}
```

Answer:

Please see page 113 for a hint.

Problem 10

Classify the form of Cohesion from the following example of code:

```
double getIncome()
{
    cout << "Enter your income: ";
    return 0.0;
}
```

Answer:

Please see page 113 for a hint.

Problem 11

Identify the type of Coupling that the following function exhibits:

```
float absoluteValue(float value)
{
    if (value >= 0)
        return value;
    else
        return -value;
}
```

Answer:

Please see page 117 for a hint.

Problem 12

Identify the type of Coupling that the following function exhibits:

```
char adjustedGrade(char grade, bool cheated)
{
    if (cheated)
        return 'F';
    else
        return grade;
}
```

Answer:

Please see page 117 for a hint.

Problem 13

Draw a Structure Chart to convert Fahrenheit to Celsius. The program has three functions besides main():

```
float getTemperature();
float convert(float fahrenheit);
void display(float celsius);
```

Answer:

Please see page 112 for a hint.

Problem 14

Draw a Structure Chart to represent the following problem: prompt the user for his hourly wage and number of hours worked. From this, compute his weekly pay (taking time-and-a-half overtime into account). Deduct from his pay his tax and tithing. Finally, display to the user how much money he has left to spend:

Please see page 112 for a hint.

Problem 15

Sue wants to write a program to help her determine how much money she is spending on her car. Specifically, she wants to know how much she spends per day having the car sit in her driveway and how much she spends per mile driving it. This program will take into account periodic costs such as devaluation, insurance, and parking. It will also take into account usage costs such as gas, repair costs, and tires. Draw a Structure Chart to represent this program.

Please see page 112 for a hint.

Assignment 2.0

Your assignment is to create three Structure Charts representing how you would solve three programming problems. In these examples, you are not concerned with how you would implement each individual function. Instead, you want to identify what the function will do (Cohesion), how information will pass between them (Coupling), and how the functions call each other.

Problem 1: Compute grade

The first problem is to create a Structure Chart for a program to convert a number grade (ex: 88%) into a letter grade (ex: B+). Consider the following example (input is Underlined):

```
What is your grade in percent: 88
Your grade is B+
```

Problem 2: Compute tithing

The second problem is designed to help a child set aside part of his allowance for tithing. This program will prompt the user for his allowance, figure out how much is left after tithing is taken out, and display the results.

```
What is your allowance? $10.50
You get to spend: $9.45
```

Problem 3: Currency

The final problem is designed to help an international traveler convert his money to various currencies. After prompting him for the amount to be converted, the program will display how many British pounds, Euros, or Japanese Yen he will have:

```
How much money do you want to convert? $100.00
British Pounds: £61.50
Euros: €70.09
Japanese Yen: ¥8079.06
```

Please bring these Structure Charts into class on a sheet of paper (face-to-face students) or take a picture and submit it electronically (online students). Don't forget to put your name on your assignment!

Please see page 112 for a hint.